TFRA-YOLO: Textile Feature Refinement and Reassembly YOLO for Closed-Loop Fabric Defect Inspection on Circular Knitting Machines

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, and Chunhua He

***Abstract*—Circular knitting machines are foundational production infrastructure in the textile industry. A fabric defect arising during production can propagate during continuous knitting and cause substantial fabric loss if it is not detected and handled promptly. To address this issue, we design and implement a cloud-edge collaborative inspection system integrating a Hall sensor, adjustable supplementary lighting, an industrial high-definition camera, an RK3566 edge terminal, a textile-specific detector, and an IoT cloud platform. When the Hall sensor indicates that the machine is operating, the edge terminal activates the camera and supplementary light. Once a defect is detected, it stops the machine through the local wired control path and reports the defect to the cloud platform for defect statistics and production-quality analysis. The proposed TFRA-YOLO starts from YOLOv8n and combines directional texture encoding (DTE) with local texture-guided feature reassembly (LTFR) to retain directional knitted-texture cues and adaptively reconstruct the feature pyramid. On images collected from a real textile production line, TFRA-YOLO achieves 66.3% AP and 96.9% AP50, improving YOLOv8n by 7.0 and 2.8 percentage points, respectively. The proposed system provides a practical route from delayed post-production inspection toward traceable, in-process defect containment for circular knitting.**

***Index Terms*—Industrial defect detection, YOLO, deep learning, edge computing, Internet of Things.**

I. Introduction

CIRCULAR knitting machines are foundational production infrastructure in the textile industry: they continuously manufacture the knitted fabrics used in T-shirts, sportswear, and other daily garments. The latest International Textile Manufacturers Federation (ITMF) statistics report 28,000 global shipments of large circular knitting machines in 2024 [1]. Conventional inspection is commonly performed after production. Consequently, a defect arising early in production may continue propagating as the machine operates, leading to substantial fabric waste before it is discovered. An effective inspection system must therefore detect and localize defects during machine operation with sufficiently low latency to enable timely intervention before defective fabric continues to accumulate.

Single-stage YOLO detectors are particularly attractive for this setting because they jointly provide real-time inference and explicit defect localization. YOLO-based detectors have been adopted for industrial surface inspection of steel strips, steel surfaces, and large stamped parts [2]–[4]. In fabric-defect detection, existing studies have adapted YOLO through lightweight or tiny-model designs [5]–[7], adaptive or deformable feature extraction [8], [9], and enhanced multi-scale fusion or prediction for weak and small defects [10]–[16]. These studies establish the suitability of the YOLO route for bounding-box localization under industrial latency and resource constraints. However, most existing modifications primarily strengthen generic feature representations rather than explicitly exploiting the directional texture structure of circular-knitted fabrics.

For circular-knitted fabric inspection, two texture-specific limitations remain in conventional YOLO feature encoding and top-down reconstruction. First, circular-knitted fabrics exhibit highly repetitive texture patterns with dominant horizontal and vertical orientations. Defects typically disrupt this regular texture in horizontally extended, vertically extended, or compact local forms such as holes. Standard convolutional feature extraction can learn these patterns implicitly, but it does not explicitly introduce an orientation-aware inductive bias for separately modeling directional and compact texture disruptions. Second, nearest-neighbor upsampling in the top-down feature pyramid reconstructs higher-resolution features with a content-independent copying rule rather than adapting reconstruction weights to local texture structure. Such content-independent reconstruction may inadequately preserve narrow defect boundaries or weak local disruptions embedded in repetitive texture. These observations motivate explicit directional texture encoding during feature extraction and content-adaptive reconstruction during top-down fusion.

Based on this motivation, we develop TFRA-YOLO to introduce texture-specific inductive biases at both feature encoding and feature reconstruction stages. To address the first limitation, we introduce a directional texture encoding (DTE) module after selected backbone stages. DTE separately models horizontal, vertical, and compact local texture evidence, while residual fusion retains the baseline representation and augments it with directional texture cues. To address the second limitation, we introduce a local texture-guided feature reassembly (LTFR) pyramid that replaces fixed nearest-neighbor top-down upsampling with location-specific feature reassembly. TFRA-YOLO preserves the lightweight single-stage YOLO detection paradigm while introducing directional texture encoding in the backbone and content-adaptive reconstruction in the top-down feature pyramid.

Beyond detector design, practical deployment on circular knitting machines requires inspection to be synchronized with machine rotation and to support low-latency local intervention. We therefore integrate TFRA-YOLO into a cloud-edge in-

K. Tan, J. Zhang, J. Yu, and C. He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China. H. Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China. Corresponding author: C. He (e-mail: hechunhua@pku.edu.cn). E-mails: K. Tan (tankangzhao@mails.gdut.edu.cn), J. Zhang (zhangjiarui1@mail2.gdut.edu.cn), J. Yu (yujunjie2@mails.gdut.edu.cn), H. Wu (heng.wu@foxmail.com), and C. He (hechunhua@pku.edu.cn).

spection system that combines image acquisition synchronized with machine rotation, RK3566-based edge inference, and cloud-based IoT monitoring. The latency-critical intervention loop remains entirely local, while defect records and device states are synchronized with the cloud IoT platform for status visualization and production-quality analysis.

The main contributions of our work are as follows.

- We introduce a directional texture encoding (DTE) module that explicitly models horizontal, vertical, and compact local texture cues and fuses them with the baseline backbone representation, providing a lightweight texture-specific inductive bias for circular-knitted defect detection.
- We develop a local texture-guided feature reassembly (LTFR) pyramid that replaces fixed nearest-neighbor upsampling with location-adaptive reconstruction, enabling content-adaptive reconstruction of local defect structures during top-down multi-scale fusion.
- We implement a cloud-edge inspection prototype that keeps the latency-critical intervention loop local while supporting centralized defect statistics and device supervision.

II. Method

A. TFRA-YOLO Architecture

TFRA-YOLO is built on YOLOv8n [17] and consists of three main components: a directional texture encoding backbone, a local texture-guided reassembly pyramid, and a multi-scale detection head. YOLOv8n is selected as the baseline because it provides favorable inference efficiency on the target RK3566 platform while retaining sufficient architectural flexibility for texture-specific modification. [18] As shown in Fig. 1, the directional texture encoding backbone introduces DTE after selected C2f stages to strengthen the representation of horizontally extended, vertically extended, and compact local texture disruptions. The local texture-guided reassembly pyramid replaces the fixed nearest-neighbor upsampling operations with LTFR, allowing higher-resolution features to be reconstructed according to local content before multi-scale fusion. The fused pyramid features are then forwarded to the detection head for defect classification and localization. The two proposed components target different stages of feature processing. DTE focuses on extracting direction-sensitive texture cues from the repetitive knitted background, whereas LTFR focuses on reconstructing and propagating these cues across feature scales.

B. Directional Texture Encoding

The visual cue of a defect in circular-knitted fabric is often a local disruption of repetitive texture rather than an isolated object boundary. Such disruptions commonly appear in horizontally elongated, vertically elongated, or compact local forms. A standard C2f block can learn these patterns implicitly through local feature aggregation, but it does not explicitly separate directional and compact texture responses. DTE is therefore introduced to encode these complementary structures with horizontal, vertical, and local branches. For a C2f output $X \in \mathbb{R}^{C\times H\times W}$, DTE first forms three depthwise branch responses:

$$F_h = \mathrm{DW}_{1\times 7}(X), \tag{1}$$

$$F_v = \mathrm{DW}_{7\times 1}(X), \tag{2}$$

$$F_l = \mathrm{DW}_{3\times 3}(X). \tag{3}$$

The 1×7 and 7×1 branches enlarge the receptive field along the horizontal and vertical directions, respectively, while the 3×3 branch preserves compact local texture information. The three responses are concatenated and fused by a 1×1 convolution:

$$F_d = \phi\left([F_h, F_v, F_l]\right), \tag{4}$$

where $[\cdot]$ denotes channel concatenation and ϕ denotes pointwise convolution followed by normalization and activation. Depthwise filtering keeps the directional branches lightweight, while the pointwise operation mixes the branch responses across channels. The fused response is added to the original C2f feature through a residual connection:

$$Y = X + F_d. \tag{5}$$

This preserves the baseline representation while introducing additional direction-sensitive texture cues with limited computational overhead. Although DTE is related to asymmetric convolution [19], it differs in using depthwise horizontal, vertical, and local branches together with three-way fusion and residual integration within the C2f backbone.

C. Local Texture-Guided Feature Reassembly

In the feature pyramid, fixed nearest-neighbor upsampling reconstructs higher-resolution features using a content-independent copying rule, without adapting to local texture structure. This can be suboptimal for multi-scale fusion in repetitive knitted textures, where narrow defect boundaries may occupy only a small portion of the local region. LTFR therefore replaces the two fixed upsampling operations with content-adaptive feature reassembly.

LTFR adopts content-aware feature reassembly to reconstruct each output location from a locally predicted neighborhood. This formulation follows the content-aware reassembly principle introduced in CARAFE [19]. Given $X \in \mathbb{R}^{C\times H\times W}$, a 1×1 compressor first reduces the kernel-prediction feature to $C_r = \max(C/4, 16)$ channels:

$$X_c = E_{1\times 1}(X). \tag{6}$$

A 3×3 encoder then predicts s^2k^2 kernel logits, where the scale factor is $s = 2$ and the reassembly kernel size is $k = 5$:

$$K' = E_{3\times 3}(X_c). \tag{7}$$

Pixel shuffle expands the logits to the upsampled grid,

$$\widehat{K} = \mathrm{PS}(K'), \tag{8}$$

and a softmax across the k^2 kernel positions produces a distinct normalized reassembly kernel at each output location:

$$K_{i,j} = \mathrm{softmax}(\widehat{K}_{i,j}). \tag{9}$$

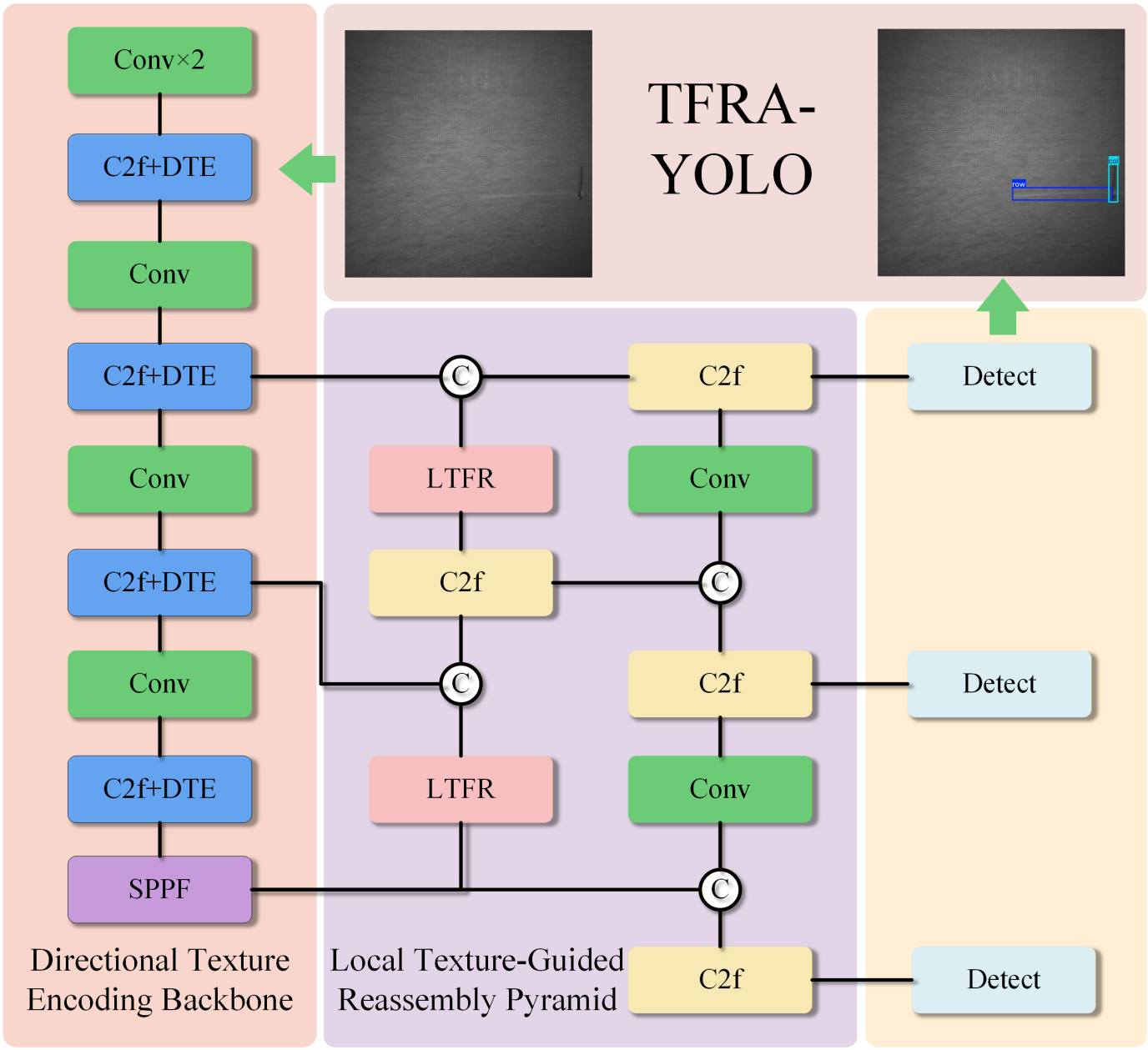

Fig. 1. Architecture of TFRA-YOLO. The Knitted Texture Fusion Backbone uses C2f–DTE blocks to encode horizontal, vertical, and local texture cues. The Local Texture Reassembly Pyramid replaces the two top-down fixed upsampling operations with LTFR, while the bottom-up path and three-scale detection head follow YOLOv8n.

The normalization constraint is

$$\sum_{u=1}^{k^2} K_{i,j,u} = 1. \tag{10}$$

Here, $E_{1\times1}$ and $E_{3\times3}$ denote the compression and kernel-encoding layers, respectively, and PS denotes pixel shuffle. In parallel, a local $k \times k$ feature patch is extracted from X and aligned with the enlarged grid:

$$P_{i,j} = \mathrm{Unfold}_{k\times k}(X)_{i,j}. \tag{11}$$

where $P_{i,j} \in \mathbb{R}^{C\times k^2}$ denotes the local feature neighborhood associated with output location (i, j). The reassembled feature is then obtained by the location-specific weighted sum

$$Y_{i,j} = \sum_{u=1}^{k^2} K_{i,j,u} P_{i,j,u}. \tag{12}$$

Unlike a spatially shared interpolation kernel, Eq. (12) allows the reconstruction weights to vary with local texture content, so defect boundaries and regular knitted regions can be aggregated differently. The channel-compression stage bounds the overhead of predicting these kernels, while the 5×5 neighborhood provides more local support than a copied source pixel. In TFRA-YOLO, this reassembly mechanism is applied specifically to the two feature-pyramid upsampling

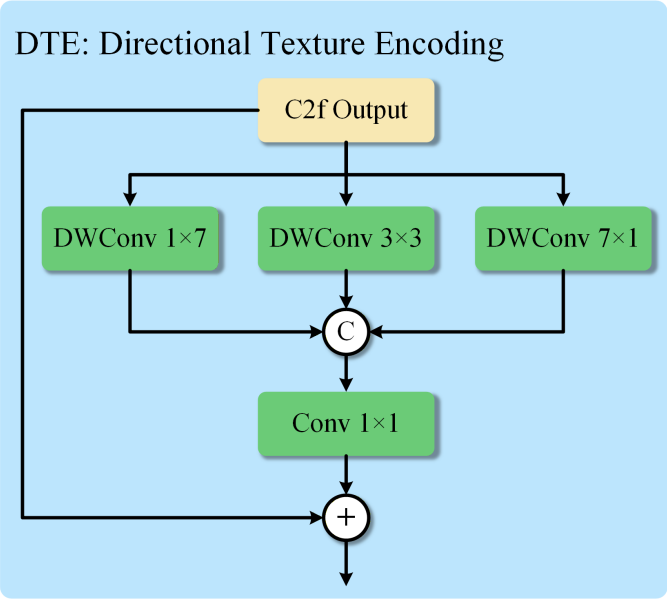

Fig. 2. DTE structure. A C2f feature is processed by horizontal, vertical, and local depthwise branches, followed by channel concatenation, 1×1 fusion, and residual addition.

stages to preserve local defect structure during multi-scale fusion.

DTE and LTFR play complementary roles: DTE enhances direction-sensitive texture representation in the backbone, while LTFR performs content-adaptive reconstruction of these features across pyramid scales.

D. Closed-Loop Inspection System

a) Production workflow.: Fig. 4 shows the inspection system installed on a circular knitting machine, which comprises a Hall sensor, an industrial camera, and an RK3566 edge device. The workflow begins when the operator presses the machine's native start button, initiating rotation. The Hall sensor detects the rotation and signals the RK3566, which activates LED illumination, acquires video frames from the camera, and begins real-time defect inspection. Once a defect is detected, the RK3566 issues a stop command via a wired connection to the machine controller, halting production before the defect propagates further. The corresponding defect frame is displayed locally for on-site operator inspection and troubleshooting and uploaded to the cloud platform for defect statistics and production-quality analysis. After the operator resolves the issue and restarts the machine, the production cycle resumes. All defect-to-stop decisions are made locally, without relying on cloud communication.

b) On-machine deployment.: The deployed edge terminal uses a Rockchip RK3566 processor running Android 13. Images are acquired with a SmartSens 1.3-megapixel industrial high-definition global-shutter camera at 1280×1024 resolution and a nominal 30 frames/s. Adjustable supplementary lighting is used to stabilize the local imaging condition. The camera is installed 10 cm from the fabric, providing an approximately 16×14 cm field of view. On the deployed terminal, the reported inference time is 45 ms per processed image (approximately 22.2 processed frames/s). This rate is sufficient for the maximum stated machine condition: a 38-inch circular knitting machine at 30 revolutions/min has a circumference of approximately 3.03 m and a fabric-surface speed of approximately 1.52 m/s. Consequently, the surface advances approximately 6.8 cm during one 45-ms inference interval, which remains below the 14-cm shorter side of the field of view and yields at least 7.2 cm nominal overlap between adjacent processed views. The edge terminal is connected to the circular knitting machine control system by a direct wired interface; the measured time from a defect decision to transmission of the stop-control signal is below 10 ms. The edge terminal synchronizes non-latency-critical status and defect records with the cloud IoT platform through MQTT. The system is designed to support continuous 24/7 inspection. Table I summarizes the configuration.

TABLE I
ON-MACHINE DEPLOYMENT CONFIGURATION.

Item	Configuration or measured value
Edge terminal	Rockchip RK3566; Android 13
Camera	SmartSens 1.3-megapixel industrial HD global-shutter camera
Supplementary light	Adjustable illumination
Acquisition	1280×1024; nominal 30 frames/s
Installation geometry	10 cm camera-to-fabric distance; approximately 16×14 cm field of view
Inference time	45 ms per processed image (approximately 22.2 processed frames/s)
Maximum-speed coverage	38-inch diameter and 30 revolutions/min: 1.52 m/s surface speed; 6.8 cm advance per processed view
Control interface	Direct wired connection to the circular knitting machine controller
Decision-to-command latency	< 10 ms from defect decision to stop-control signal transmission
Cloud communication	MQTT-based synchronization
Cloud platform	Digital twin: live state, defect analytics, and authorized remote control
Operating mode	Designed for continuous 24/7 inspection

c) Platform-aware deployment.: The choice of YOLOv8n as the baseline is driven by deployment compatibility rather than by version chronology alone. The RKNN Model Zoo provides tested RK3566 NPU examples and single-core INT8 throughput figures for several YOLO families at 640×640 input [18]. In the current official benchmark, YOLOv8n reaches 34.0 frames/s, exceeding the listed newer YOLOv10n (20.7 frames/s) and YOLO11n (20.6 frames/s). The legacy YOLOv5n and YOLOv6n entries are faster (39.7 and 48.8 frames/s, respectively), which directly demonstrates that a newer YOLO version is not automatically a better NPU choice. We therefore select YOLOv8n as a high-throughput baseline with tested RK3566 NPU examples in the official Model Zoo and an extensible structure for textile-specific modification, rather than claiming that it is the absolute fastest model in the official benchmark. The benchmark values are used only for platform-aware model selection: they do not substitute for measuring the final customized detector on the deployed application.

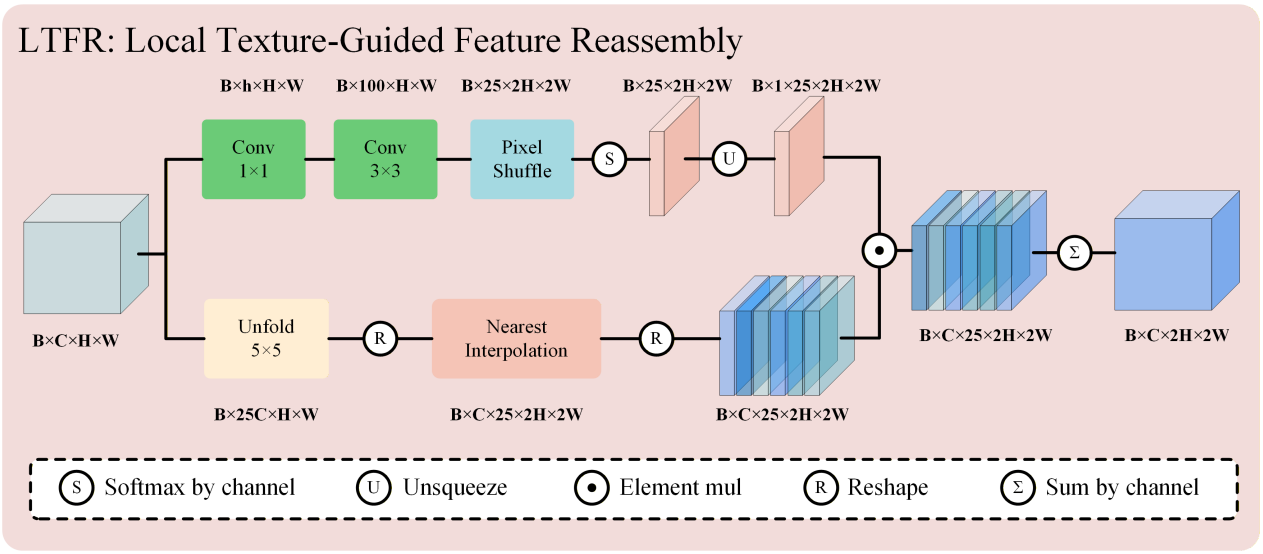

Fig. 3. LTFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds and aligns local feature patches. Their element-wise product is summed along the kernel dimension to produce a $2\times$ upsampled feature.

Training produces a PyTorch checkpoint, which is exported at 640×640 through ONNX and then converted to an RKNN model for the RK3566 NPU. We use the official RKNN-Toolkit2 conversion workflow, which targets Rockchip NPUs and supports model conversion, quantization, inference, and performance evaluation [20]. Integer-only quantization is a well-established route to reduce inference cost on constrained hardware [21]. In the deployment path, post-training INT8 quantization is applied during ONNX-to-RKNN conversion with representative training images used for calibration; the resulting RKNN artifact is loaded by the Android edge application for NPU execution. This workflow is summarized as *PyTorch checkpoint* $\rightarrow$ *ONNX* $\rightarrow$ *INT8 RKNN* $\rightarrow$ *RK3566 NPU*. The reported 45-ms edge inference time in Table I corresponds to the deployed application, not PyTorch inference on the training workstation.

Fig. 5 shows the corresponding cloud IoT platform. The platform provides a portal view, defect-count trend statistics, device status and remote configuration, and a review page for uploaded defect records. These cloud functions support supervisory operation and production-quality analysis; they are separated from the latency-critical local wired stop path.

The design follows the edge-intelligence principle that time-sensitive perception and action remain near the physical process, while selected evidence can be retained centrally [22]. The cloud side is organized as a manufacturing digital-twin service for live state visualization, statistics, and authorized supervision [23], [24]. Here, the control consequence is more direct: the event is escalated to a worker, not accepted as a final autonomous verdict. This provides a pragmatic route for handling rare defects while retaining the records needed to identify recurring defect patterns and production-quality issues.

d) Operational boundaries.: The workflow separates three stages: (i) whether machine rotation enables inspection, (ii) whether the detector produces a defect decision and initiates the local stop path, and (iii) how an operator inspects and troubleshoots the stopped-machine event. The reported recognition experiments validate only the second stage on a held-out image split. The deployed configuration establishes geometric image coverage at the stated maximum machine speed, the camera, edge terminal, wired control interface, and decision-to-command latency, but does not yet quantify physical machine-stop time, false-stop rate on defect-free fabric, frame-delivery or drop statistics during continuous operation, or operator-troubleshooting completion rate. These measurements should be reported from a controlled site trial before submission.

III. Experimental Setup

A. Dataset

The dataset was acquired on a real circular-knitting production line using the deployed camera arrangement described in Section II-D. Frames containing visible defects were retained and annotated with bounding boxes by the project team. The three engineering labels are *row*, *col*, and *hole*; *row* and *col* typically manifest as horizontal and vertical texture disruptions, respectively. The supplied annotations define an image-level train/validation/test split of 282/40/81 images, as summarized in Table II. The held-out test set contains 91 labeled defects. Production-roll, material-batch, and temporal identifiers were not retained with the released annotation package; therefore, the present evaluation is reported as an image-level split and does not claim explicit isolation of adjacent video frames. This limitation is considered when interpreting the results and motivates a future production-period evaluation with grouped splits.

B. Training and Evaluation

Table III summarizes the implementation protocol used for the Ultralytics experiments. TFRA-YOLO and its direct

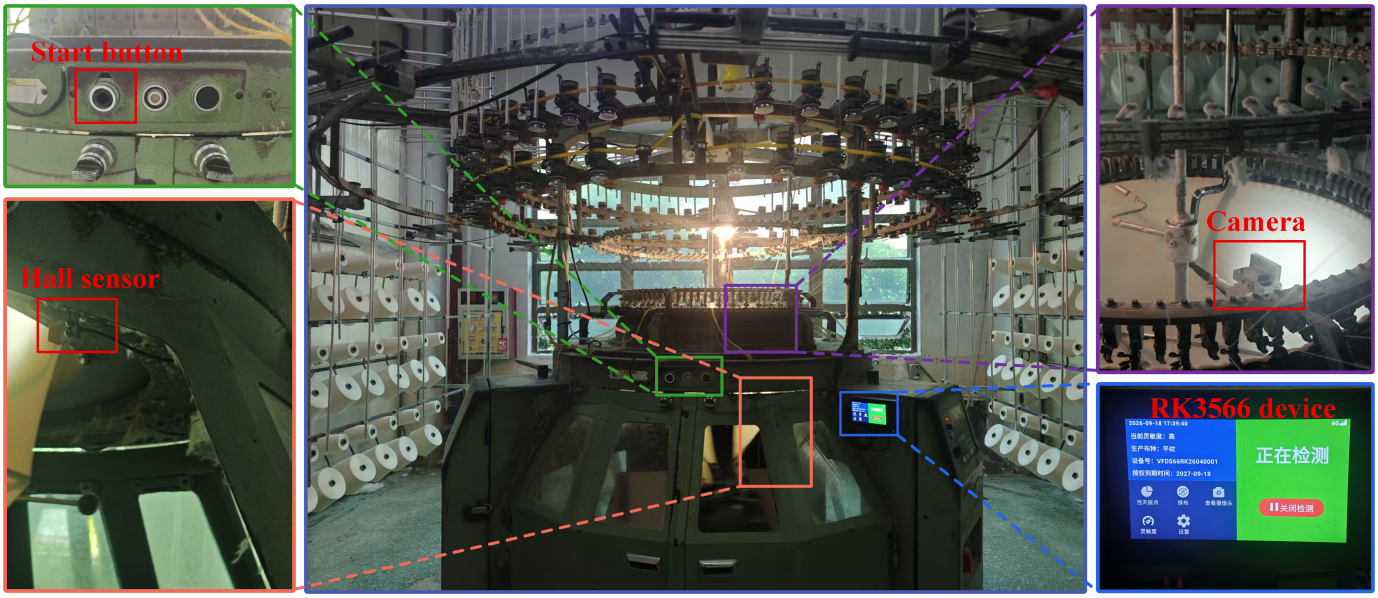

Fig. 4. Actual deployment on the circular knitting machine. The insets highlight the start button, Hall sensor, industrial camera, and RK3566 edge device, respectively.

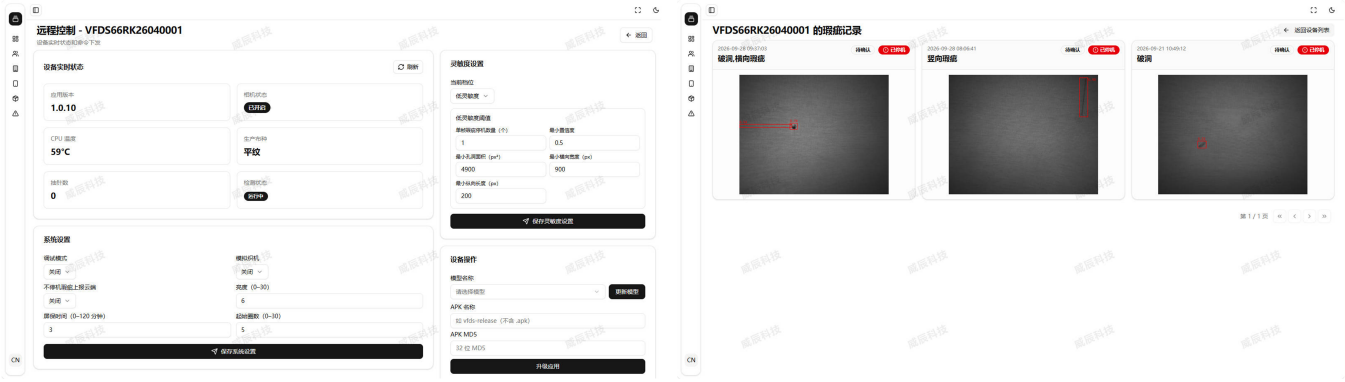

Fig. 5. Cloud IoT platform for the deployed inspection system. The four panels show the platform portal, defect-count trend dashboard, device-state and remote-configuration interface, and uploaded defect-record review interface, respectively. Cloud communication supports status visualization, statistics, and authorized supervision, whereas the defect-to-stop control path remains local and wired.

TABLE II
IMAGE-LEVEL SPLIT AND DEFECT-INSTANCE COUNTS IN THE REAL PRODUCTION-LINE DATASET.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

YOLOv8n baseline are initialized from `yolov8n.pt`; the remaining comparison models use the available official pre-trained initialization in their respective repositories. Evaluation is conducted on the held-out test split with a maximum of 16 detections per image.

C. Evaluation Metrics

We use the COCO detection protocol [25]. For a predicted box B_p and a ground-truth box B_g, intersection over union (IoU) is defined as $\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g|/|B_p \cup B_g|$. At an IoU threshold t, a prediction is counted as a true positive only when it has the correct class and is matched one-to-one to an as-yet unmatched ground-truth instance; all remaining predictions are false positives and unmatched ground-truth instances are false negatives. For each class c, $\mathrm{AP}_c(t)$ is the area under its interpolated precision–recall curve. With $C = 3$ classes, the reported AP and AP50 are

$$\mathrm{AP} = \frac{1}{10C} \sum_{c=1}^{C} \sum_{t \in \mathcal{T}} \mathrm{AP}_c(t),$$
$$\mathrm{AP50} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{AP}_c(0.50), \tag{13}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$. Thus, AP measures localization and classification quality across increasingly strict overlap thresholds, whereas AP50 emphasizes detection at IoU $= 0.50$. Both are written as percentages in the tables. We additionally report trainable parameters and FLOPs at $[1, 3, 640, 640]$ to characterize model scale and computational

TABLE III
IMPLEMENTATION PROTOCOL AND TRAINING HYPERPARAMETERS.

Item	Specification
Training hardware	NVIDIA GeForce RTX 3090 GPU (24 GB)
Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Model initialization	TFRA-YOLO and YOLOv8n: `yolov8n.pt`; other baselines: the available official pretrained checkpoint in the corresponding repository
Input resolution	640×640
Batch size / data workers	16 / 8
Maximum epochs / early stopping patience	300 / 80 epochs
Optimizer	Ultralytics automatic optimizer selection
Data augmentation	Standard Ultralytics geometric and photometric augmentations; no augmentation hyperparameter was manually overridden
Checkpoint retained for testing	Framework-selected best validation checkpoint
Test protocol	Held-out test split, maximum 16 detections per image

cost; these are not substitutes for measured edge-device latency.

For each model, the retained best-validation checkpoint is evaluated once on the same held-out test split. FLOPs and parameter counts are reported at $[1, 3, 640, 640]$. The values are suitable for a coarse efficiency comparison, but cross-framework FLOP accounting can omit unsupported operators; no cross-framework latency claim is made. DETR, Deformable DETR, Faster R-CNN, and RT-DETRv2 follow their respective repositories and pretrained backbones [26]–[29].

IV. RESULTS AND DISCUSSION

A. Comparison With Detectors

Table IV compares TFRA-YOLO with the retained baselines. TFRA-YOLO achieves 66.3% AP and 96.9% AP50. Relative to its direct YOLOv8n baseline, AP improves by 7.0 points and AP50 by 2.8 points for 0.38 M additional parameters and 0.8 G additional FLOPs. Among the compact YOLO variants evaluated here, TFRA-YOLO has the highest AP. RT-DETR-R18 obtains 67.7% AP, 1.4 points higher than TFRA-YOLO, but requires 20.09 M parameters and 60.72 G FLOPs. This is a useful trade-off rather than a claim that TFRA-YOLO dominates every detector.

Fig. 6 visualizes the parameter- and operation-accuracy trade-off without shifting any data coordinates. The position of TFRA-YOLO in the low-parameter region is consistent with the tabulated result. Because the horizontal axes are logarithmic, the visual is intended to show scale rather than imply a linear performance-efficiency relationship.

B. Ablation

Table V isolates the two modifications within the same YOLOv8n family. DTE contributes the larger isolated improvement (+4.8 AP), while LTFR contributes +1.6 AP. Combining both gives 66.3% AP, which is 0.6 points above the arithmetic sum of the isolated gains. This indicates that the texture representation and adaptive reassembly are complementary: DTE supplies direction-specific texture cues, whereas LTFR transfers them through the top-down path. The current study establishes module-level complementarity; branch-level DTE and alternative-upsampling ablations are required before making more granular claims about individual directional branches, the directional kernel length, or the choice of content-adaptive reassembly.

C. Per-Class and Qualitative Results

The TFRA-YOLO checkpoint obtains 61.7%, 73.8%, and 63.4% AP for *row* (horizontal), *col* (vertical), and *hole* defects, respectively. The notably high *col*-defect AP should be interpreted with care because the test category has only 12 instances. Fig. 7 presents seven selected images and the available standardized renderings. The visual comparison is illustrative rather than a replacement for the full quantitative test set. It is particularly useful for examining thin *col*-type defects, long *row*-type defects, and holes against repetitive background texture.

Fig. 8 further compares the activation visualizations produced by the official Ultralytics visualization routine for the YOLOv8n baseline and TFRA-YOLO. Each row corresponds to the same test image, with the original image, baseline visualization, and TFRA-YOLO visualization arranged from left to right. These maps are used only as qualitative evidence of where the detectors respond; the AP results in Tables IV and V remain the basis for the quantitative comparison.

D. Limitations and Submission-Ready Validation

The present evidence supports a promising lightweight detector on a real production-line image set, but it does not yet establish production reliability. First, the test split is small and category-imbalanced, and its image-level partition does not establish isolation of adjacent video frames. Second, FLOPs are not a substitute for end-to-end edge latency, especially when image capture, preprocessing, postprocessing, control communication, and physical stopping are part of the operational loop. Third, the current ablation isolates the two modules but does not yet separately quantify each DTE branch or compare LTFR with alternative interpolation and reassembly operators. Finally, operator troubleshooting records can support defect statistics and production-quality analysis, but their completeness and operational value have not yet

TABLE IV
TEST-SET COMPARISON ON 81 IMAGES AND 91 INSTANCES. AP IS COCO AP@[.50:.95]. ALL VALUES ARE PERCENTAGES EXCEPT PARAMETERS AND FLOPS.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
YOLOv5n	2.50	7.10	93.1	58.4
YOLOv8n	3.01	8.10	94.1	59.3
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	96.8	64.3
Faster R-CNN-R50	41.13	181.45	96.1	64.7
TFRA-YOLO	3.39	8.90	**96.9**	66.3

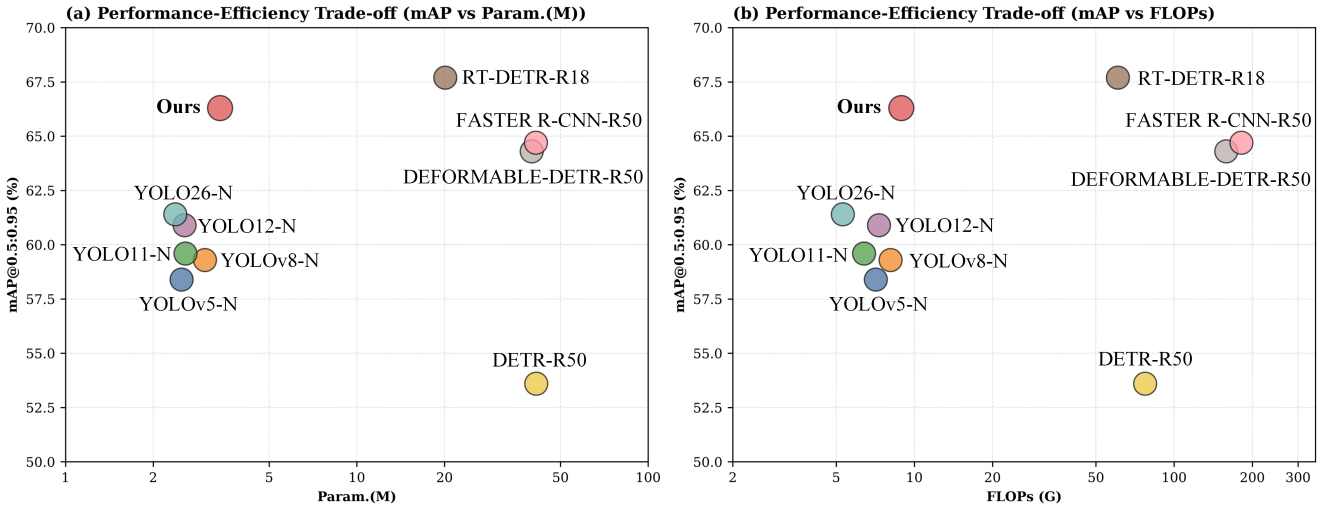

Fig. 6. Accuracy-efficiency trade-off of the retained test results. Both horizontal axes use a logarithmic scale. Marker coordinates are the exact values in Table IV; the plot does not contain manual coordinate compression.

TABLE V
ABLATION ON THE COMMON HELD-OUT TEST SPLIT USING THE SAME YOLOV8N TRAINING RECIPE.

Variant	DTE	LTFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ DTE	✓	–	3.28	8.8	94.2	64.1
+ LTFR	–	✓	3.11	8.2	94.3	60.9
TFRA-YOLO	✓	✓	3.39	8.9	96.9	66.3

been measured. These points define a concrete validation plan, rather than weaknesses that should be concealed.

V. CONCLUSION

This work presents a closed-loop framework for defect inspection on a circular knitting machine and TFRA-YOLO, a compact direction- and content-adaptive detector. The workflow couples rotation-aware edge inspection, automatic local wired stop signaling, on-site operator inspection and troubleshooting, and a cloud digital twin for live status visualization, defect statistics, production-quality analysis, and authorized remote supervisory control. The detector combines DTE for horizontal, vertical, and local knitted-texture encoding with LTFR for adaptive feature-pyramid reconstruction. On the retained test split, TFRA-YOLO improves YOLOv8n from 59.3% to 66.3% AP at 3.39 M parameters and 8.9 G FLOPs. The next step is not merely more offline accuracy: it is a disciplined site evaluation of sensing-to-stop latency, false-stop behavior, operator-troubleshooting workflow, production-quality-analysis value, and grouped generalization across production conditions.

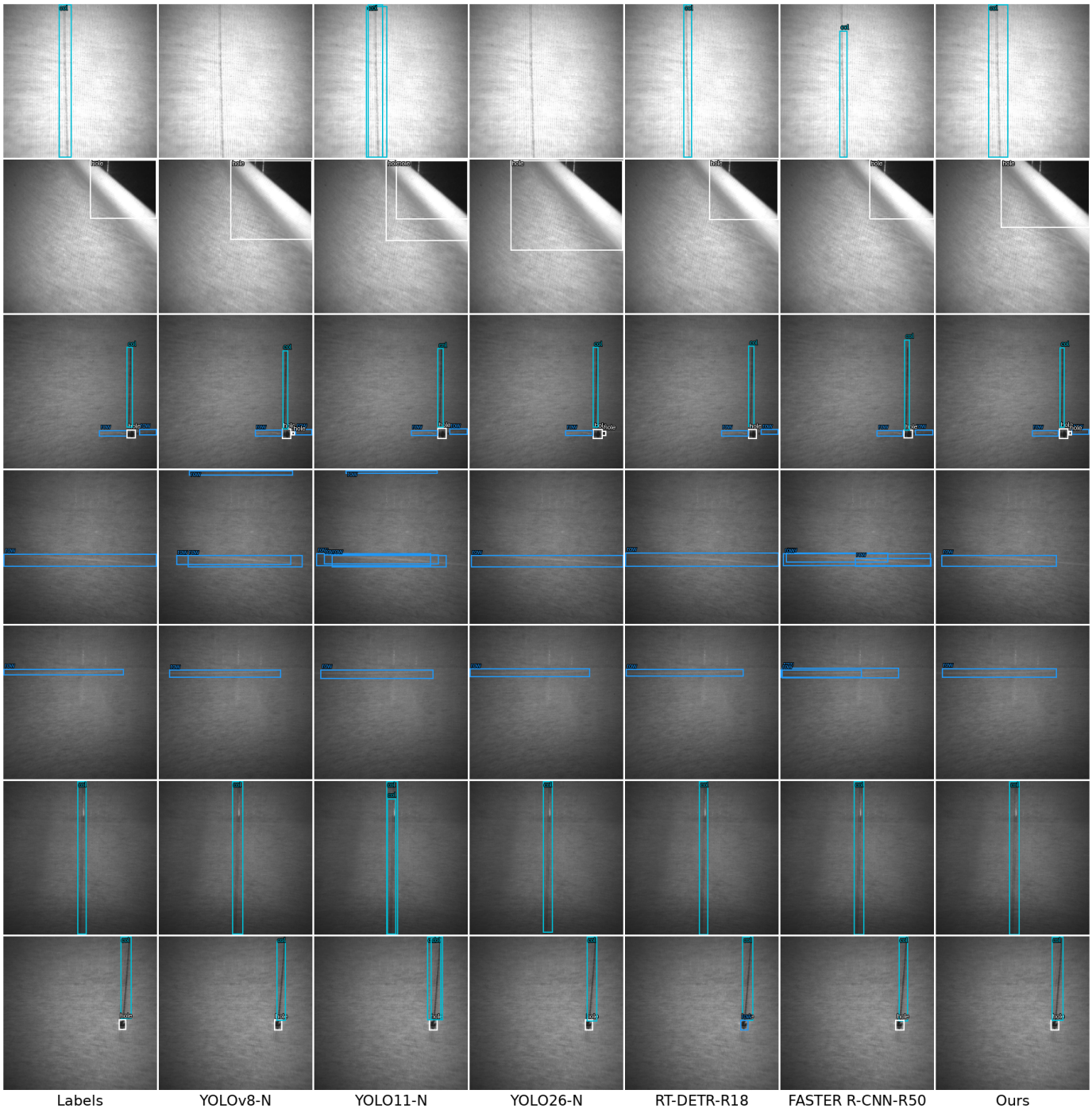

Fig. 7. Qualitative comparison on selected test images. Columns are labels, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and TFRA-YOLO. The figure uses a common renderer for visual consistency; it should be updated after finalizing the exact sample-selection protocol and display confidence threshold.

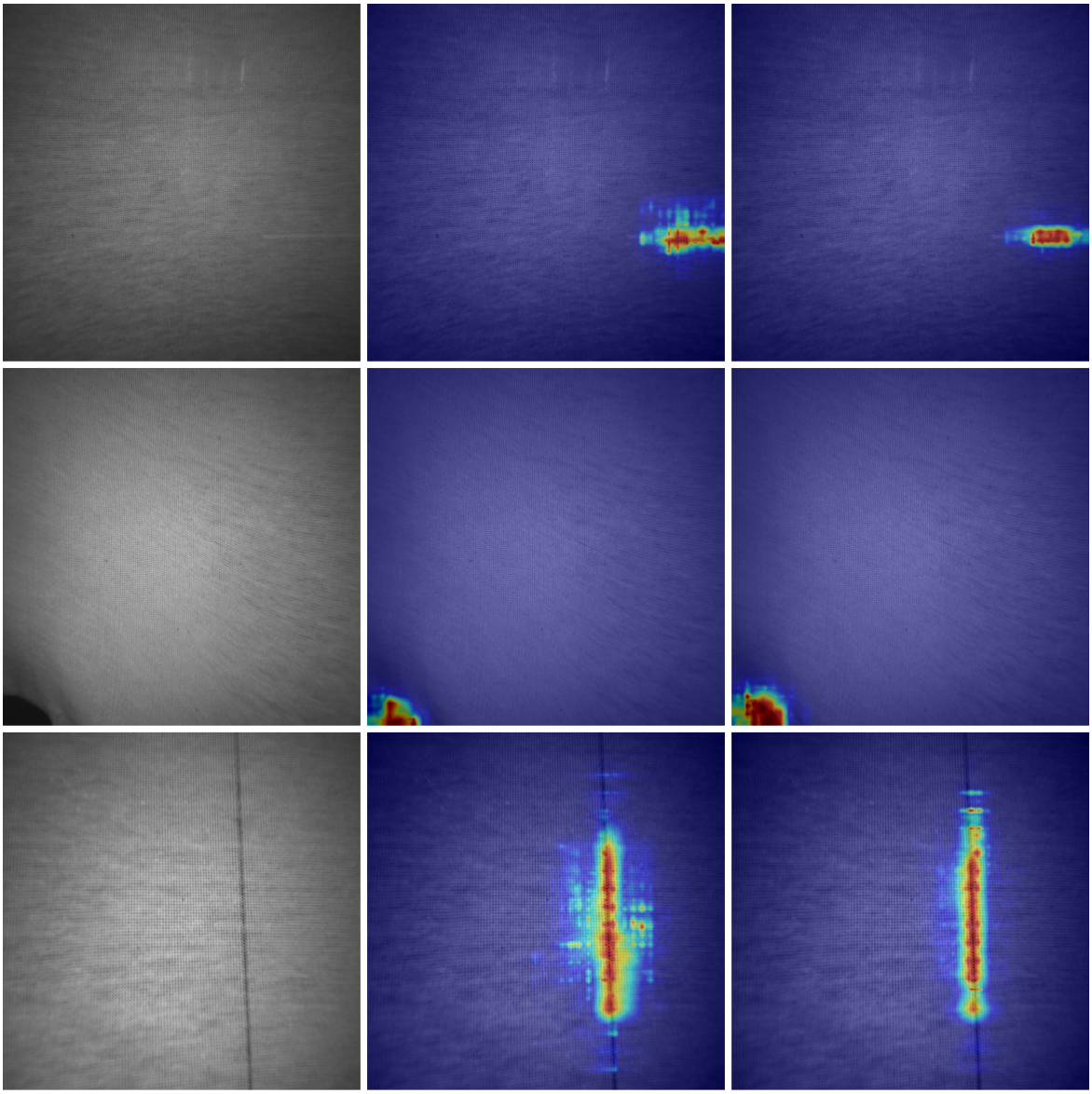

Fig. 8. Qualitative activation visualization on three selected test images. The left, middle, and right columns show the original images, YOLOv8n visualizations, and TFRA-YOLO visualizations, respectively. Each visualization is generated with the same official Ultralytics routine.

REFERENCES

[1] International Textile Manufacturers Federation, "International textile machinery shipment statistics (ITMSS), vol. 47/2024," 2025, [Online]. Available: https://www.itmf.org/publications/statistics-publications/international-textile-machinery-shipment-statistics-itmss-2024.

[2] S. Gao, M. Chu, and L. Zhang, "A detection network for small defects of steel surface based on YOLOv7," *Digital Signal Processing*, vol. 149, p. 104484, 2024.

[3] C. Li, A. Xu, Q. Zhang, and Y. Cai, "Steel surface defect detection method based on improved YOLOX," *IEEE Access*, vol. 12, pp. 37 643–37 652, 2024.

[4] X. Dong, C. Zhang, J. Wang, Y. Chen, and D. Wang, "Real-time detection of surface cracking defects for large-sized stamped parts," *Computers in Industry*, vol. 159, p. 104105, 2024.

[5] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Text. Res. J.*, vol. 94, pp. 225–245, 2024.

[6] H. Yang, J. Jing, Z. Wang *et al.*, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Text. Res. J.*, vol. 94, pp. 49–68, 2024.

[7] C. Chen, Q. Zhou, L. Xiao *et al.*, "LWFDD-YOLO: A lightweight defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 9–10, pp. 1125–1142, 2025.

[8] X. Luo, Z. Cheng, Q. Ni *et al.*, "Defect detection algorithm for fabric based on deformable convolutional network," *Textile Research Journal*, vol. 93, no. 9–10, pp. 2342–2354, 2023.

[9] Z. Zhang, T. Huang, W. Zhang *et al.*, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, 2025.

[10] X. Yue, Q. Wang, L. He, Y. Li, and D. Tang, "Research on tiny target detection technology of fabric defects based on improved YOLO," *Applied Sciences*, vol. 12, no. 13, p. 6823, 2022.

[11] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, no. 6, pp. 890–898, 2024.

[12] C. Chen, Q. Zhou, S. Li, D. Luo, G. Tan *et al.*, "Fabric defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 3–4, pp. 235–251, 2025.

[13] L. Zhou, B. Ma, Y. Dong, Z. Yin, and F. Lu, "DCFE-YOLO: A novel fabric defect detection method," *PLOS ONE*, vol. 20, no. 1, p. e0314525, 2025.

[14] Z. Gao, S. Zhang, J. Su, B. Li, J. Wang, and Z. Song, "Research on improved YOLOv8 fabric surface defect detection algorithm," *Textile Research Journal*, vol. 96, no. 17–18, 2026.

[15] S. Xu, C. Wan, W. Feng, J. Li, and J. Gao, "YOLO-DLL: An effective detection method for fabric defects," *Textile Research Journal*, 2025.

[16] T. Wang *et al.*, "SRSE-YOLO: An improved fabric defect detection method for YOLOv8," *Textile Research Journal*, 2025.

[17] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[18] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[19] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[20] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[21] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[22] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[23] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.

[24] W. Kritzinger, G. Karner, C. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[25] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

[26] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[27] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[28] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[29] W. Lv, Y. Xu, Y. Yang, G. Li, J. Wang, L. Chen *et al.*, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.